CricketMind

Verified Claims over Real Evidence

Thesis Architecture Draft — Version 4

Status. This is a design whose first part is built. Built and tested so far (days 1–13 of a 202-day plan): the claim language, that is, provenance classes, subjects, claims that check their own rules when they are created, findings with slots, the renderer, and the registry with its start-up check, with 83 unit tests; and a first literature search. Nothing else has been built or measured. Every number in this document is a design parameter, a figure cited from the literature, or an illustrative computation on the synthetic corpus of the v2 reference implementation, and is labelled as such. The accuracy of the perception front end belongs to a separate study and is not reported here. The codebase is still called `cricketmind-v3`; this is version 4 of its architecture document.

1 What Changes in Version 4, and Why

Version 2, described in the ICCIT paper [1], established the control flow: two entry paths, two bounded correction loops, twelve stakeholder agents, and a Faith Check that compares every cited figure against two sanctioned engines. Version 3 closed the four gaps its evaluation exposed: the gate checked values rather than claims, four agents asserted claims no engine computed, predictions were verified as if they were facts, and the evidence was synthetic.

Version 3 was then reviewed against a stricter target, a journal paper, and against work published in 2026 that also checks language-model numbers against tool outputs [16, 17, 18, 19]. Five gaps remain, and Version 4 is organised around closing them.

1. **Correct numbers can still be framed misleadingly.** Checks C1–C6 verify every number and the names around it, but not the judgment the prose wraps around them: "Kohli struggles with the cover drive: {c1}" passes when {c1} is a strong strike rate.

2. **Questions arrive in any wording, and most are on no list.** Version 3 answered the questions its nine claim types anticipated. Users word the same question in many ways, and ask about rules, about things no data records, and about combinations nobody listed.

3. **A question can itself be false, or hostile.** "Why is Kohli bad against spin?" takes a weakness for granted; answering it as asked repeats the error with apparent authority. A question can also carry instructions ("ignore your rules").

4. **A reader cannot re-check a number.** A provenance tag says where a number came from, not how to reproduce it.

5. **The method is shown in one sport, and the planned evaluation would not convince a journal.** It compared v3 only with v2, on questions its author wrote, with no outside test set, no ablation, no alternative verifier and no human judgment.

Table 1 lists the changes. The principle is kept and extended: numbers and kinds of claims still come only from sanctioned sources; now judgments must be backed by checked comparisons, and every answer must be honest about what it could check.

#	Change	Motivation	Where
1	Tone check C7, with fabrication class F8	Misleading framing (gap 1)	Faith Check C7; registry `sr_vs_average`; N8 M1
2	Five kinds of question, a read-back, filters × measures, the Explainer, an honest decline	Wording and coverage (gap 2)	N3, N8 M1, N10, N11m, N12, N13
3	False premises and trick attempts	False or hostile questions (gap 3)	N3; C7
4	A receipt for every number	Numbers cannot be reproduced (gap 4)	C2; N13; a receipt checker
5	A sport-neutral core and a football case study	One sport only (gap 5)	Subject; one registry, engine set and gazetteer per sport
6	A journal-grade evaluation	Weak evidence (gap 5)	Section 10
R	Self-checking claims; halves round up; an exported classifier with a model card	The first build days; the classifier study	Claim; renderer; N1–N5

Table 1: Changes from Version 3. Node names are kept; N11m, the Explainer, is new.

2 Design Invariants

Version 4 keeps the six invariants of Version 3 and adds three. Every later section is an implementation of one of them.

I1 — Numeric authority.
Only three sources may produce a number: N6 (*measured* from the clip), N8 (*descriptive*, counted from history) and N9 (*predictive*, simulated). N9 is code and never becomes a language-model call.

I2 — No hollow claims.
Every claim type an agent may assert resolves to a metric of one sanctioned source. This is checked when the graph is built, so a template that asserts an unsupported kind of claim cannot be deployed.

I3 — Agents never type numbers.
Agents write prose with claim references; a deterministic renderer inserts the verified values.

I4 — Uncertainty travels with the claim.
Predictive and comparative claims carry their interval, and their wording must fit it.

I5 — No measurement without a clip.
A text-only query can never carry a measured claim; the posture fields are absent, not defaulted.

I6 — Bounded cost.
Every loop is bounded (at most two retries, two regenerations and four active agents, the Explainer counting as one), so worst-case cost stays closed-form.

I7 — No unsupported judgment.
A tone word in an answer, or a judgment a question takes for granted, is a hidden comparison. It must be backed by a checked comparison claim whose interval lies wholly on the side of zero the words imply; otherwise the words go, or the premise is corrected.

I8 — Every number can be re-checked.
Each verified claim carries a receipt: the checksum of the data snapshot, the exact engine query and the engine's version. Re-running the receipt reproduces the number.

I9 — Every question gets an honest answer.
The answer first states how the question was read. It then proves what the engines can compute, explains rules in labelled words without numbers, and otherwise says plainly that the question cannot be checked. It never invents.

3 System Architecture Diagram

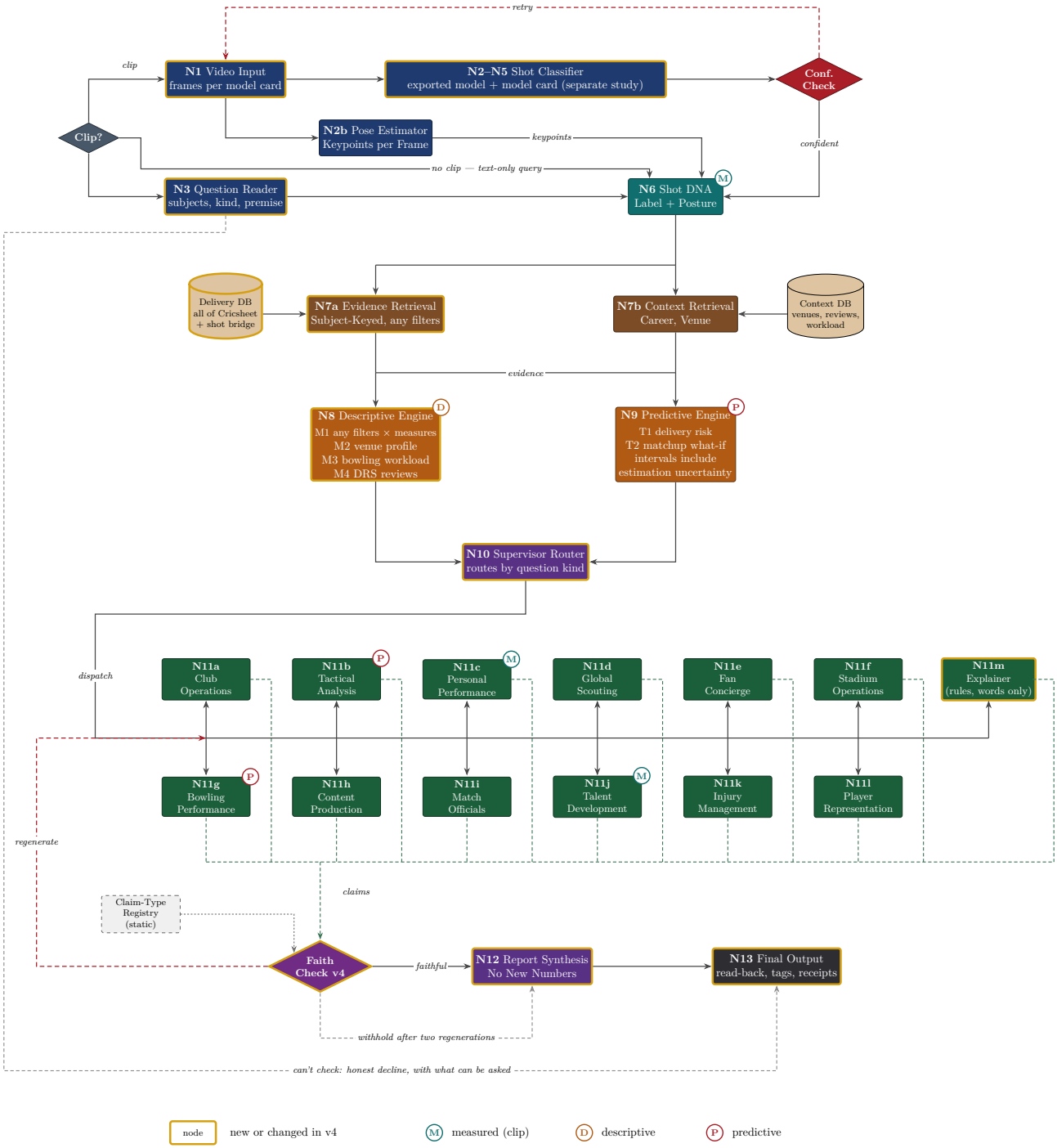


Figure 1: CricketMind LangGraph architecture, Version 4. Gold outlines mark nodes that are new or changed since v3. Badges give the provenance class a node produces or an agent may cite: M for measured from the clip, D for descriptive (counted from history), P for predictive (simulated). N3 gives every question one of five kinds (Table 2). A question nothing can check goes straight to the output with an honest decline; for a rules question the evidence and engine nodes pass the state through unchanged and N10 sends it to the Explainer, N11m. Agents emit claim tuples rather than numbers; the Faith Check re-queries the engines for each tuple, keeps the receipt of that re-query, and runs checks C1–C7 (Figure 3) against the static claim-type registry. Failing agents are regenerated with the reasons, at most twice, after which the finding is withheld and listed in the report. The classifier N2–N5 is one exported model from a separate study, and N1 picks frames exactly as its model card states. Data sources and the offline pipeline are shown in Figure 2.

4 Answering Any Question Honestly

N3 keeps the structured parse of Version 3 and also gives every question one of five kinds (Table 2). The kind decides the path, and the path decides what kind of answer is possible.

Kind	Example	Path	Answer
Stats	"Kohli's batting average against Australia since 2020?"	N7a, N8 M1	checked facts, tagged D
Prediction	"The chance Kohli is out within six short balls of pace in the powerplay?"	N7a, N9	checked, with its interval; the wording must fit its band (C4)
Video	"Is this a cover drive? Is his weight forward?"	N1–N5, N2b, N6	the label with its confidence; measured posture, tagged M
Rules	"What is LBW?"	the Explainer, N11m	words only, labelled "general explanation, not checked against data"
Can't check	"Is Kohli injured?"	none	"I can't check that", and what can be asked instead

Table 2: The five kinds of question. Every answer starts with a read-back of how the question was understood.

The read-back. Every answer begins "I read this as: . . . ", listing the parsed player, measure, filters and scope, for example "I read this as: V Kohli; batting average; against Australia; since 2020; all formats." A misreading then shows on the screen instead of silently answering a different question.

Filters times measures. In Version 3 each claim type was, in effect, one fixed question. In Version 4, N8 M1 computes any registered *measure* (batting: runs, strike rate, average, dismissal rate, boundary and dot-ball share; bowling: wickets, economy, bowling strike rate) under any combination of supported *filters* (player, bowler, opposition, venue, season, format, innings, phase, bowler type, batting hand). The registry lists measures, not questions, so a small registry answers a very large number of questions, each computed by the engine and checked by C2. A new measure is one registry row, one engine function and one line of renderer words. Language-model-written SQL is deliberately not used: on cricket questions most such SQL executes while little of it returns the right answer [36], and nothing would check it.

Paraphrases. Different wordings of one question should parse to the same subjects and measure, and so reach the same engine query and the same verified answer. This is tested directly: the locked exam asks questions in several wordings (Section 10).

The Explainer, N11m. Rules and general questions go to a new node that answers in words only. Its answer is labelled "general explanation, not checked against data"; C5 still forbids digits, apart from a small allowlist ("T20", "U-19"); and it may not name players, since any statement about a player needs data. A checked rules source, such as the MCC Laws of Cricket, is future work.

The honest decline. If no engine could answer (injuries, salaries, private medical or contract data), N3 marks the question *can't check* and the output says so, with examples of what can be asked instead. No agent is woken, and nothing is invented.

5 Node-by-Node Changes

5.1 Perception through an Exported Model: N1–N5

In Version 3, N2, N4 and N5 named an EfficientNetV2-S frame encoder, a temporal transformer and a fifteen-class head. The classifier is now built in a separate study, CricShot, and reaches CricketMind

as one exported model with a model card that states exactly what input it expects. CricketMind never imports CricShot code; the choice of backbone and its accuracy belong to that study.

Frame selection is part of this contract. CricShot's preprocessing drops repeated frames (some clips carry copies inserted by frame-rate conversion), finds the batter, keeps the camera shot with the most detections, crops to it, and only then takes evenly spaced frames from what remains. Fifteen evenly spaced frames from the raw clip would feed the model inputs it never saw in training, and no error would be raised. N1 therefore follows the model card, and a parity test compares CricketMind's frames and crops with CricShot's on a fixed set of clips: frame numbers exactly, pixels within a tolerance. The confidence check and its two retries are unchanged.

5.2 N2b Pose Estimator

Unchanged from Version 3. A pose model [7] extracts seventeen body keypoints per frame on the same frames, and three posture metrics are computed: weight transfer, stride length, and head over front knee, each scaled so that it does not depend on camera distance. Posture has no ground truth, so each metric is validated by how well it separates shots played mainly off the front foot from shots played mainly off the back foot (ROC-AUC on the CricShot10k test split, threshold 0.75, a design parameter). A metric that fails leaves the registry. The test uses dataset labels, not the classifier, and reports no classifier accuracy.

5.3 N3 Question Reader

N3 keeps Version 3's structured parse under a fixed schema (batter, bowlers, shots, phase, bowler type, venue, format, and an optional scenario), with every name resolved against the player register and an unknown player reported as such. It adds three outputs: the *kind* of question (Section 4); the *read-back*; and any *premise*, a judgment the question takes for granted. "Why is Kohli bad against spin?" presupposes that he is worse than average against spin. A marked premise becomes a comparison the agents must cite and C7 must check (Section 7). Text inside a question is only ever read as a question, never as an instruction to the system.

5.4 N6 Shot DNA

Unchanged. N6 fuses the accepted shot label, the measured posture (provenance M, video path only, invariant I5) and the parsed subjects into one record. Downstream nodes never see a pixel.

5.5 Evidence Layer and Retrieval

O1, O2, O4: ball-by-ball evidence. O1 flattens every Cricsheet competition into one row per delivery and records the snapshot's date and checksum, which every receipt names. O2 attaches batting hand and bowling type through the player register, so that filters such as "against pace" have a source. O4 derives the tables the N8 modules read: venue aggregates, balls bowled per bowler per date, and the DRS reviews attached to deliveries.

O3: the commentary-to-shot bridge. Unchanged from Version 3. Ball-by-ball commentary names the stroke ("drives through cover"), but no labelled dataset pairs commentary with the fifteen CricShot10k classes. O3a labels each line weakly [8]: keyword rules first, then, for lines the rules cannot resolve, a language model constrained to the fifteen classes or *none*. O3b measures agreement with a stratified hand-labelled sample of about 500 lines by Cohen's κ [9] and per-class F1; the bridge is accepted at $\kappa \geq 0.61$, "substantial" agreement [10]. O3c joins labels to deliveries by match, innings, over and ball; each label keeps its confidence and origin. The language model produces categories that are validated before use, never a number an agent can cite, so invariant I1 is untouched.

N7a and N7b. N7a filters by any combination of the supported filters and returns the matching deliveries with their count. A shot filter applies only to deliveries the bridge labelled. N7b retrieves career and venue context as before.

Source	Provides	Does not provide	Access and caveats
CricShot10k [2]	15-class batting-shot clips	identity, outcome	held locally
Cricsheet [3], every competition, men's and women's	every delivery: batter, bowler, runs, extras, wicket, venue, date; DRS reviews	shot type, line and length, bowler type	free with attribution; the snapshot's date and checksum are recorded; review data has needed corrections [4]
Player register with Cricinfo attributes [5]	batting hand, bowling type	—	identifiers link the two; check terms of use
Commentary corpus	text that names the stroke played	shot labels; match and ball keys unverified	no clean, labelled public corpus was found; must be licensed
Open football event data [6]	every pass, shot and tackle of the matches covered	—	case study only (Section 8); check the licence; pin a snapshot
Injury, replay, sponsorship, eligibility, salary records	nothing public	—	questions about them get an honest decline

Table 3: Evidence sources.

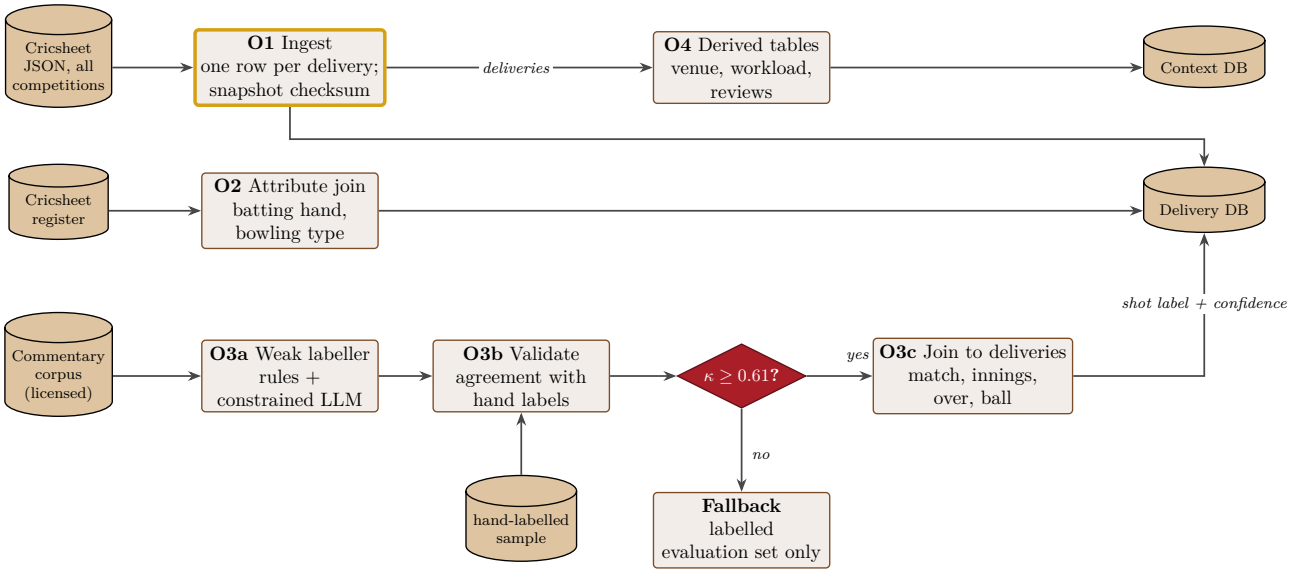


Figure 2: Offline evidence pipeline. Nothing here runs at query time. The commentary bridge (O3) is accepted only if its labels agree substantially with a hand-labelled sample; otherwise shot-keyed statistics are restricted to a hand-labelled evaluation set.

5.6 N8 Descriptive Engine

N8 stays deterministic aggregation over a pinned data snapshot: every output can be recomputed by hand, and every receipt reproduces it.

M1 — delivery statistics, any filters × measures.
The measures and filters of Section 4, with sample sizes; differences between two subjects with a bootstrap 95% interval (used by C4); and `sr_vs_average`, a player's strike rate against the average batter under the same filters, with a bootstrap interval (used by C7).

M2 — venue profile
(backs N11f). Average first-innings score, share of matches won chasing, and the share of wickets taken by pace and by spin.

M3 — bowling workload
(backs N11k). Balls bowled in the last 7 and 28 days and their ratio. Workload spikes were

associated with roughly double the injury risk in the following week in elite fast bowlers [11], but the ratio's validity is disputed [12], and match data omits training loads. M3 therefore emits a *monitoring flag*, never an injury probability.

M4 — DRS reviews
(backs N11i). Review counts and overturn rates by team. Cricsheet keeps no appeal kind, and the dismissal is known only when the batter ended up out, so a rate by dismissal type would be lopsided.

5.7 N9 Predictive Engine and Honest Uncertainty

T1 and T2. Unchanged from Version 3. Tier 1 answers the delivery question ("six short balls: what is the chance of a wicket?"). Tier 2 answers matchup counterfactuals ("what if this bowler bowled the over instead of that one?"), estimated from head-to-head rates shrunk toward population rates [13]. A counterfactual is always reported as a difference against a baseline, with an interval; if the interval contains zero, the output is *no detectable difference*.

Two kinds of uncertainty. v2's interval measured only Monte Carlo error, which shrinks as trials increase; it omitted estimation uncertainty, which shrinks only with more data. On the synthetic corpus (illustrative only), the batter's pull has 4 dismissals in 50 balls:

	P(out within 6 balls)	**95% interval**
v2: Monte Carlo error only (20,000 trials)	0.394	[0.387, 0.400]
since v3: estimation uncertainty propagated	0.405 (median)	[0.155, 0.697]

The second row draws the per-ball rate from a Beta posterior (Jeffreys prior) and pushes each draw through the simulation. The honest interval is about twenty times wider, and it spans three wording bands of Table 6.

Calibration. T1 and T2 are calibrated on a held-out season: reliability diagrams and the Brier score [14].

5.8 N10 Supervisor Router

The router keeps Version 3's evidence summary, its capability filter, the cap of four active agents and the default pair. It now also routes by kind: stats, prediction and video questions go to the stakeholder agents, and rules questions go to the Explainer, which counts as one of the four.

5.9 The Agents

Stakeholder	Agent	Claim types it may assert	Prov.
Franchises / Boards	Club Operations (N11a)	`strike_rate`, `dismissal_rate`, `venue_first_innings_avg`, `venue_chasing_win_share`, `venue_wicket_share`, `team_win_share`	D
Coaches	Tactical Analysis (N11b)	`strike_rate`, `dismissal_rate`, `sr_difference`, `sr_vs_average`, `p_dismissal_within_k`, `matchup_difference`	D, P
Players	Personal Performance (N11c)	`strike_rate`, `sr_difference`, `sr_vs_average`, `runs`, `batting_average`, `boundary_share`, `dot_ball_share`, `weight_transfer`	D, M
Scouts	Global Scouting (N11d)	`strike_rate`, `sr_vs_average`	D
Fans	Fan Concierge (N11e)	`strike_rate`, `runs`, `batting_average`, `team_win_share`, `match_won_by_runs`, `match_won_by_wickets`	D
Stadiums / curators	Stadium Operations (N11f)	`venue_first_innings_avg`, `venue_chasing_win_share`, `venue_wicket_share`	D
Bowlers / bowling coaches	Bowling Performance (N11g)	`wickets`, `economy`, `bowling_strike_rate`, `matchup_difference`	D, P
Broadcasters / sponsors	Content Production (N11h)	`strike_rate`, `sr_difference`, `sr_vs_average`, `boundary_share`	D
Governing bodies / umpires	Match Officials (N11i)	`review_overturn_rate`	D
Academies	Talent Development (N11j)	`strike_rate`, `sr_difference`, `sr_vs_average`, `weight_transfer`	D, M
Medical teams	Injury Management (N11k)	`workload_ratio_flag`	D
Player agents	Player Representation (N11l)	`strike_rate`	D
Anyone asking about rules	**Explainer (N11m)**	none: words only, labelled, no digits, no player names	—

Table 4: The Version 4 roster: Version 3's twelve agents and the Explainer, in the menu's own names. Every claim type in the third column is a registry entry backed by an engine metric (invariant I2), and every agent's tone words must now be backed by a checked comparison (C7).

A figure for a ground, a season or a recent period is one of these measures narrowed by a filter from the question: career, venue and recent strike rate are all `strike_rate`, so an agent cannot choose its own window. A bowler's rate of taking wickets is `bowling_strike_rate`, and a boundary figure is a share of balls. Every count carries its sample size, so sample size is not a claim type of its own. The workload flag's sample is the balls bowled in four weeks; the week's balls appear only inside its ratio.

6 The Claim Contract

An agent finding is a list of claims and a prose template. The claim is the unit of verification:

```
{"id": "c1", "kind": "descriptive", "metric": "strike_rate", "engine": "N8.M1",
 "subject": {"player": "V Kohli", "shot": "Cover Drive", "phase": "powerplay",
             "bowler_type": "pace"},
 "value": 142.0, "n": 50}
{"id": "c2", "kind": "predictive", "metric": "p_dismissal_within_k", "engine": "N9.T1",
 "subject": {"player": "V Kohli", "shot": "Pull", "phase": "powerplay",
             "bowler_type": "pace", "k": 6},
 "value": 0.405, "interval": [0.155, 0.697]}
```

```
template: "Against pace in the powerplay, Kohli's cover drive returns {c1}.
           Six short balls carry {c2} of a dismissal."
rendered: "Against pace in the powerplay, Kohli's cover drive returns a strike
           rate of 142.0 [D, 50 balls]. Six short balls carry a 41% chance
           [P, range 16% to 70%] of a dismissal."
```

The values are the synthetic examples of Section 5.7, and the rendered line is the output of the built renderer.

Claims check themselves (built). A claim refuses to exist if it breaks one of four rules: its identifier is `c` followed by digits, so templates can reference it; a descriptive claim carries its sample size, $n \geq 1$; a predictive claim carries a 95% interval that contains its value; a measured claim names its clip (I5). The wording band of a predictive claim is not stored, as Version 3's example did, but computed from the value when C4 needs it, so the two can never disagree. Claims and findings are immutable once made, and a finding keeps its claims in an immutable sequence, so nothing can be added after the check.

The renderer (built). A deterministic renderer replaces each reference with words, the verified value and a tag: `[D, 50 balls]`, `[P, range 16% to 70%]`, `[M, clip vid1_0]`. The agent never types a digit (I3). Displayed numbers round exact halves up, in decimal arithmetic on the number's shortest written form: 0.405 shows as 41%, 0.155 as 16% and 0.145 as 15%. Python's default formatting gives 40%, 16% and 14% for the same values, because it rounds an exact half to the even neighbour and because 0.145×100 falls just below 14.5 in binary floating point; a reader comparing the report with the engine would see unexplained one-point differences.

Subjects: one shared base. The checks, the claims and the renderer only ever see a `Subject`; each sport defines its own fields.

```
@dataclass(frozen=True, kw_only=True)
class Subject:                        # the checks only ever see this
    pass

@dataclass(frozen=True, kw_only=True)
class CricketSubject(Subject):
    player: str | None = None
    shot: str | None = None
    phase: str | None = None
    bowler_type: str | None = None
    # bowler, other_bowler, other_shot, venue, match_format,
    # team, dismissal_type, date, k
```

Fields are keyword-only, so `CricketSubject("Pull", "V Kohli")`, which would silently swap player and shot, is refused; subjects are frozen, so a checked claim's subject cannot change. A football subject is another subclass, and nothing in the checks changes (Section 8).

Receipts. When C2 re-queries the named engine for a claim's subject, the gate keeps that query as the claim's receipt: the checksum of the pinned data snapshot, the exact engine query and the engine's version. N13 shows the receipt with the number, and a receipt checker re-runs any receipt and confirms the value. Receipts come from the gate's own re-query, never from the agent.

The claim-type registry. Table 5 shows the registry at its first build. It is built with the graph, and two things are checked then: every claim type an agent declares exists, and every registry entry resolves to an engine metric. A template that asserts "the surface is holding pace" with no venue metric behind it is therefore not a runtime failure but a build failure. The measures of Section 4 join it later, one row each.

Claim type	Prov.	Source	Required subject keys	Checks
`strike_rate`	D	N8.M1	player; other filters optional	C2 exact; C3 floor
`dismissal_rate`	D	N8.M1	as above	C2; C3
`sr_difference`	D	N8.M1	player, two shots	C2; C4 zero-crossing; backs C7
`sr_vs_average`	D	N8.M1	player; compared with the average batter under the same filters	C2; C3; backs C7
`venue_first_innings_avg`	D	N8.M2	venue, format	C2; C3 (matches)
`workload_ratio_flag`	D	N8.M3	bowler, date	C2; wording: flag only
`review_overturn_rate`	D	N8.M4	team or dismissal type	C2; C3
`weight_transfer`	M	N6	a clip	C2; video path only
`p_dismissal_within_k`	P	N9.T1	player, shot, phase, bowler type, k	C2 with interval; C4 band
`matchup_difference`	P	N9.T2	batter, two bowlers, phase, k	C2 with interval; C4 zero-crossing

Table 5: The claim-type registry at its first build: the nine types of Version 3 and `sr_vs_average`. The menu in the code has since grown to 22 rows, and Table 4 gives every one of them to an agent.

7 Faith Check v4

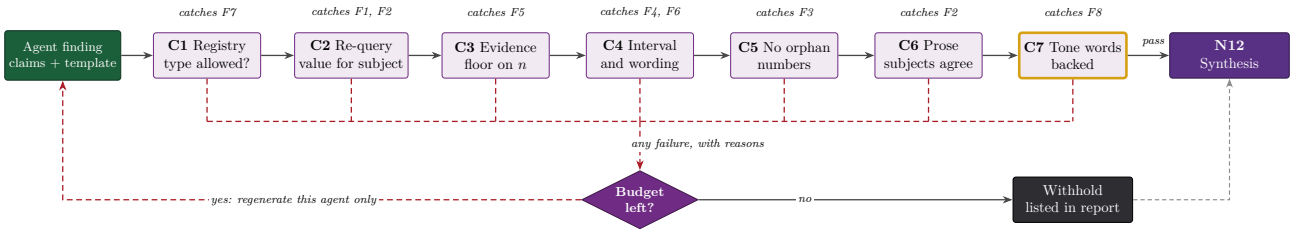


Figure 3: Faith Check v4. The seven checks run in order, cheapest first, inside the single Faith Check node. Each catches particular fabrication classes of Table 8. C1–C6 come from Version 3; C7 is new.

C1 — Schema and registry.
The finding parses; each claim type exists and this agent is allowed to assert it; the claim names its source.

C2 — Subject-bound value.
The gate re-queries the named source with the claim's subject and compares the result within rounding, including the interval for predictive claims, and keeps the query as the claim's receipt. A correct number cited for the wrong subject fails here, because the engine returns a different value for that subject.

C3 — Evidence floor.
A descriptive claim on fewer than $n_{\min}$ balls (a design parameter; default 30) must carry a small-sample qualifier, and may not be generalised ("struggles against spin") without one.

C4 — Interval and wording.
A predictive claim's wording must fall in the band of Table 6 that contains its point estimate; if the interval crosses a band boundary, a hedge naming the evidence is required. A comparative claim whose difference interval contains zero must say *no detectable difference*.

C5 — No orphan numbers.
Apart from claim references and a small allowlist ("T20", "U-19"), the template contains no digits.

C6 — Prose subjects agree.
Using a gazetteer of players, the fifteen shot names and their synonyms, phases, bowler types and

venues, every entity named in a sentence that contains a claim reference must agree with that claim's subject.

C7 — Tone words backed (new).
Tone words carry a direction ("strong", "dominates", "excellent" point up; "struggles", "weak", "vulnerable" point down). A tone word in a sentence about a player must be backed by a comparison claim in the same sentence (`sr_vs_average`, or a difference between two subjects) whose whole 95% interval lies on the side of zero the word points to. A tone word with no comparison, with one pointing the other way, or with an interval containing zero, fails.

C7 by example (an invented illustration). Suppose `sr_vs_average` for a batter's cover drive is +22 with interval $[+8, +36]$. "He is strong on the cover drive" passes. "He struggles with the cover drive" fails: the word points down, the comparison up. "He is strong on the pull" fails if the pull's comparison interval contains zero. "He struggles" with no comparison claim fails, because the judgment has no evidence behind it.

False premises and trick attempts. When N3 marks a premise ("Why is Kohli bad against spin?"), the agents must cite the comparison that tests it, and C7 checks their words against it. If the comparison does not support the premise, the answer says so before anything else; if the evidence is too thin for any judgment (C3), it says that instead. A trick attempt inside a question changes nothing: questions are never read as instructions, and every number must still pass C1–C7.

Worked example: the v2 running example. On the synthetic corpus the batter's cover drive returns a strike rate of 142.0 and his pull 112.0, both on 50 balls. The bootstrap 95% interval for the difference is $[-16.0,\ +76.0]$, which contains zero. Under C4, the v2 template's verdict that the cover drive is "the stronger stroke" is a spurious difference (class F6): the agent must report no detectable difference on this evidence. The example is illustrative, but it shows that the stricter gate changes conclusions, not only formatting.

Point estimate p	**Required wording (any of)**	**Forbidden in this band**
$p \le 0.13$	almost certainly not, remote	likely, probable, possible
$0.13 < p \le 0.40$	probably not, unlikely	likely, probable, expected
$0.40 < p < 0.60$	about even, as likely as not	likely, unlikely
$0.60 \le p < 0.87$	probable, likely	almost certain, will
$p \ge 0.87$	almost certain, very likely	certain, guaranteed
any band	—	will, certain, guaranteed, always, never

Table 6: Wording bands for predictive claims, after Kent's words of estimative probability [15], with his ranges closed into contiguous intervals. If the 95% interval crosses a boundary, a hedge naming the evidence ("on 4 dismissals in 50 balls") is also required.

On the Tier 1 example of Section 5.7, the median of 0.405 falls in *about even* and the interval crosses two boundaries, so an acceptable rendering is "about as likely as not, though on 4 dismissals in 50 balls the estimate is very uncertain". "Will probably get him out" fails C4.

N12 and N13. N12 composes the report from verified findings only and is held to the same rule: it references claims and types no numbers, and its output passes C5, C6 and C7. If it fails, the report falls back to the verified findings in fixed order. N13 shows the read-back, every number with its provenance tag and receipt, any explanation with its label, and the withheld findings.

8 Generality: a Sport-Neutral Core

A method shown in one sport could be a cricket trick. Version 4 separates what any sport shares from what each sport supplies (Table 7), and tests the split with a small football case study.

Shared by every sport (unchanged for football)	Supplied per sport
Provenance classes; `Claim` and its four rules; findings and slots; the renderer's rounding and tags; the registry mechanism and its build-time check; Faith Check C1–C7; receipts; the graph and its bounds; the five kinds of question	the `Subject` subclass and its fields; registry rows; engines; renderer words; the gazetteer used by C6; the tone-word list used by C7; agent prompts; the data pipeline; perception (cricket only)

Table 7: What is shared and what each sport supplies.

The football case study. Open football event data [6] replaces Cricsheet; about ten measures (goals, shots, conversion, passes, pass completion, tackles, and one prediction with an honest interval) replace the cricket measures; three agents (a coach, a scout and a fan) replace the twelve; and `FootballSubject` carries player, team, opponent, competition, season and minute. Video is not ported: football event data already records what every event was. The study reports the number of changed lines in the checks (the target is zero), the 8-lie test on football findings, and a locked football exam of 40 questions. It is a case study, not a second product.

9 Control Loops and Bounds

The two loops of Version 3 are unchanged in structure. The retry loop guards against perception errors; the regenerate loop re-runs only the failing agents and passes them the failing checks and reasons. The Explainer counts toward the cap of four active agents, so agent invocations remain bounded by $4 \times (1 + 2) = 12$, independent of roster size. The premise is marked inside N3 and checked inside the Faith Check, receipts are kept by C2's existing re-query, and C7 runs inside the single Faith Check node, so no new loop or node is added on the agent path. A can't-check question skips the agents entirely. The superstep bound must still be re-derived from the closed-form cost model and re-measured before it is claimed.

10 Evaluation Plan

Protocol. The paper's final numbers come from a *locked exam*: 100 questions across the five kinds (20 written four different ways each, plus 20 outside every list in the plan) and 20 tricky questions (false premises and trick attempts), each with its expected handling. The student writes it, ideally with others writing in their own words, before the system can answer anything; its SHA-256 checksum is recorded, the system's builder never reads it, and it is run once. Improvements are made only on a separate practice set (40 questions written five ways, 30 outside the lists, and 30 rules questions from SportQA [37]), which is never reported as a result. The football case study has its own locked exam of 40 questions. All sizes are design parameters.

Fabrication containment. The injection harness covers eight fabrication classes (Table 8). The v2 column follows from the v2 code, which compares cited values only; it is a prediction to be confirmed, not a measurement.

Statistics. Every reported rate carries a confidence interval; every comparison between systems carries a significance test; anything random (seeds, language-model sampling) is run several times; and labels made by hand report inter-rater agreement.

Class	Injected error	Example	v2 gate	v4 check
F1	wrong value	strike rate 162 instead of 142	caught	C2
F2	right value, wrong subject	one batter's 142 attributed to another	missed	C2, C6
F3	uncited number	"about 150" typed in the prose	missed	C5
F4	certainty inflation	"will likely be dismissed" at $p = 0.405$	missed	C4
F5	small-sample generalisation	"struggles against spin" on six balls	missed	C3
F6	spurious difference	"the stronger stroke" when the interval contains zero	missed	C4
F7	unsupported claim type	"the surface is holding pace" with no venue metric	missed	C1, registry
F8	unsupported tone	"struggles with the cover drive" beside a strike rate well above average	missed	C7

Table 8: Fabrication classes for the injection harness. F8 is new in Version 4; C1–C6 would also miss it.

Experiment	Question it answers	Design	Evidence row
Containment	Are lies caught?	F1–F8 injected into correct findings; v2 mode (values only) against v4; synthetic and real-data findings	E-004, E-005
Ablation	Is each check needed?	v4 with each of C1–C7 turned off in turn	E-016
AI checkers	Why not let an AI check?	an AI judge [33] and Chain-of-Verification [34] as alternative gates on the same harness: lies caught, correct findings wrongly stopped, time and cost	E-017
Cost of checking	What does checking cost?	correct findings wrongly stopped; invocations and completeness as the lie rate rises	E-005, E-006
Coverage	Does every question get an honest answer?	the locked exam: proved, explained, declined; every wrong answer; the same checked facts for every wording	E-013
Baselines	Is it better than simpler systems?	the same exam answered by a plain language model with no data, and by one given the engines' facts without the Faith Check; every stated number checked against the engines	E-014
Three models	Does it depend on one model?	two local models and one online model, with version and date recorded	E-018
Tricky questions	Are premises corrected and tricks resisted?	the 20 tricky exam questions	E-019
Outside test set	Does it hold on questions others wrote?	CricBench's English questions [36], if they are released	E-020
Generality	Does the method transfer?	football: changed lines in the checks; the 8-lie test; a locked exam	E-021
Receipts	Can every number be reproduced?	every number in the final exam re-run from its receipt	E-022
Engines	Are the engines right and calibrated?	recomputation against hand-computed cases; N9 calibration on a held-out season	E-007, E-012
Perception, bridge	Do posture and shot labels hold up?	posture front-foot against back-foot AUC; bridge κ and per-class F1	E-008, E-009
Routing, cost bound	Right agents, bounded cost?	a labelled routing set with paraphrases; measured cost against the closed-form bound	E-010, E-011
Human study	Do people trust and use the answers?	at least 10 raters (fans, players, coaches); 20 exam questions; the three systems' answers shown blind and in random order	E-015

Table 9: Experiments. The last column points to the row of the thesis evidence map that each experiment will verify.

11 Build Plan

Phase	Work	Days	Milestone
0	Foundations: environment, the five layers, an architecture test	1–5	built
1	Claims: provenance, subjects, claims, findings, renderer, registry; the literature search, the journal choice and the locked exam	6–15	days 6–11, 13 done
2	Faith Check C1–C7 and the fabrication harness	16–27	M1, day 27
3	Real cricket data: every Cricsheet competition into DuckDB	28–37	
4	Engines: N8 (any filters × measures, receipts) and N9	38–54	M2, day 54
5	The graph: all nodes, the agents and the Explainer	55–79	M3, day 79
6	The web API	80–91	
7	The website	92–111	M4, day 110
8	Video: the exported classifier and pose	112–127	M5, day 127
9	The commentary-to-shot bridge	128–142	M6, day 140
9b	The football case study	143–149	
10	Evaluation (Table 9)	150–179	M7, day 179
11	Ship it: containers, documentation, a demo	180–187	M8, day 186
12	Write the paper	188–202	M9, day 202

Table 10: Build plan (worst case, one focused session per day). Estimates, not commitments.

12 Risks and Fallbacks

Risk	Effect	Fallback
No licensed commentary corpus with match and ball keys	O3 impossible	hand-label an evaluation set; shot-keyed claims only on it
Bridge labels too noisy ($\kappa < 0.61$)	wrong shot statistics	disable the shot filter on real data; report the bridge as a negative result
A 7B model breaks the claim schema	findings do not parse	constrained decoding; parse failures count as C1 rejections and regenerate
Pose fails on low-resolution clips	no posture	metrics failing validation leave the registry; agents degrade to D claims
Workload ratio validity disputed	misleading medical advice	worded as a monitoring flag; never a probability
Bowling type missing for some players	filters silently shrink n	n is always reported; C3 floor
N3 misreads an unusual wording	a correct answer to the wrong question	the read-back makes it visible; measured on the exam's paraphrases
A tone word missing from C7's list, or sarcasm	misleading framing slips through	numbers always carry their own label; misses counted on the exam
An explanation is wrong	a wrong rules answer	labelled "not checked"; no digits or player names allowed
CricBench is not released in time	no outside test set	ask the authors early; otherwise the paper says so
Online models change over time	results may not repeat	record model, version and date; local models stay the main results
Ethics approval for the human study is slow	the study is delayed	ask on day 14, more than four months before the study

Table 11: Risks and fallbacks.

13 Positioning against Related Work

Checking a language model's numbers against a trusted source is an active area, and it moved quickly in 2026. A first systematic search (day 13: sixteen fixed queries over arXiv, the ACL Anthology, Semantic Scholar and the open web) found the following closest work. Proof-Carrying Numbers [16] binds each displayed number to a structured claim and verifies it in the renderer, so an unchecked number defaults to unverified; CricketMind's slots and renderer (I3) share this idea and do not claim it. Tool receipts [17] check an agent's claims against signed records of its tool calls, and Thucy [21] reports the SQL behind each verdict; CricketMind's receipts (I8) serve reproducibility, not forgery detection, and are not claimed either. EG-VAR [18] lets a proof kernel mark a claim verified only when it descends from an attested tool call, and abstains otherwise. VeriFin [19] grounds the operands of financial claims in filed facts, so a wrong period, unit or line item is caught, and checks the calculation with a solver. RAGShield [20] links numbers in government text to their entity. GSAR [22] scores the groundedness of claims in multi-agent reports and decides to proceed, regenerate or replan, close to the Faith Check's containment loop, and work on collective hallucination [30] studies how errors spread between agents. Against this work, Version 4 states its contributions narrowly:

1. **A subject-keyed re-query for multi-filter statistics.** Binding a number to what it is about is not new in itself (VeriFin, RAGShield, and the named-entity errors of sports summaries [26]). What remains is re-querying every claim for a subject defined by several filters at once (player, shot, phase, bowler type, ...), with the prose's names checked against it (C2, C6).
2. **Uncertainty wording gated against an external interval.** Work on faithful uncertainty [31, 32, 27] concerns a model's *own* confidence. Cerda-Mardini et al. [24] measured that models choose miscalibrated words for external probabilistic predictions; no system found enforces the fit before release (C4).
3. **Judgments as checked comparisons.** Unsupported evaluative words are a known error type: annotators marked "strong" as an incorrect word in a generated basketball summary [26], and the tone of language-model statistical reports drifts from the data [25]. No system found requires each tone word, and each premise of a question, to be backed by a comparison whose interval points the same way (C7, I7).

Two further elements are parts of the method rather than headline claims: a registry of claim types checked at start-up (I2), whose nearest relative checks tool calls against a registry at call time [23]; and honest, measured coverage (I9), which builds on the abstention literature [28].

The evaluation also compares the deterministic gate with language-model checkers, an AI judge [33] and Chain-of-Verification [34], since "why not let another model check?" is the obvious alternative. In cricket, FanCric [35] coordinates language-model agents for fantasy team selection, and CricBench [36] found that most SQL generated by language models for cricket analytics questions executes while far less returns the right answer: direct evidence that cricket numbers produced by language models must be verified. Sports question answering more broadly has benchmarks for knowledge and numeric reasoning [37, 38], and a 2026 survey of large models in sports [29] names hallucination as a barrier to trust. These verdicts rest on abstracts (and, for [26], the full text); they are re-checked against full texts and a repeated search before submission.

14 What This Document Does Not Claim

- Only the claim language and the registry (days 6–11) are built; no other Version 4 component has been built, and no Version 4 result has been measured.
- The worked examples use the synthetic corpus and say nothing about real players; examples marked as invented are illustrations only.
- The perception front end's accuracy is outside the scope of this document.
- The workload flag is not an injury predictor.
- Shot-keyed statistics on real data exist only if the commentary bridge passes validation.
- C7 checks the tone words on its list; sarcasm and subtle framing are not checked.
- Explanations of rules are not checked against any source.

- Generality is shown by one case study, not proven for every sport.
- The contributions above were checked by one systematic search, at abstract level; the search is repeated before submission.

References

[1] The CricketMind authors, "CricketMind: A verification-gated multi-agent architecture for multi-stakeholder cricket analytics," submitted to the International Conference on Computer and Information Technology (ICCIT).

[2] M. K. Dihan, Abdullah, Amina, and S. Ahmed, "CricShot10k: A large-scale video dataset for cricket shot classification," *IEEE Access*, 2026, doi:10.1109/ACCESS.2026.3663220.

[3] S. Rushe, "Cricsheet: Freely-available structured data for cricket," `https://cricsheet.org`.

[4] S. Rushe, "Review corrections," Cricsheet, `https://cricsheet.org/article/review_corrections/`.

[5] R. J. Hyndman *et al.*, "cricketdata: International cricket data," R package, `https://pkg.robjhyndman.com/cricketdata/`.

[6] StatsBomb, "StatsBomb open data," `https://github.com/statsbomb/open-data`.

[7] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023, `https://github.com/ultralytics/ultralytics`.

[8] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré, "Snorkel: Rapid training data creation with weak supervision," *Proc. VLDB Endowment*, vol. 11, no. 3, pp. 269–282, 2017.

[9] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[10] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[11] B. T. Hulin, T. J. Gabbett, P. Blanch, P. Chapman, D. Bailey, and J. W. Orchard, "Spikes in acute workload are associated with increased injury risk in elite cricket fast bowlers," *British Journal of Sports Medicine*, vol. 48, no. 8, pp. 708–712, 2014.

[12] F. M. Impellizzeri, S. Woodcock, A. J. Coutts, M. Fanchini, A. McCall, and A. D. Vigotsky, "What role do chronic workloads play in the acute to chronic workload ratio? Time to dismiss ACWR and its underlying theory," *Sports Medicine*, vol. 51, no. 3, pp. 581–592, 2021.

[13] B. Efron and C. Morris, "Data analysis using Stein's estimator and its generalizations," *Journal of the American Statistical Association*, vol. 70, no. 350, pp. 311–319, 1975.

[14] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[15] S. Kent, "Words of estimative probability," *Studies in Intelligence*, vol. 8, no. 4, pp. 49–65, 1964.

[16] A. V. Solatorio, "Proof-Carrying Numbers (PCN): A protocol for trustworthy numeric answers from LLMs via claim verification," arXiv:2509.06902, 2025.

[17] A. Basu, "Tool receipts, not zero-knowledge proofs: Practical hallucination detection for AI agents," arXiv:2603.10060, 2026.

[18] J. Ren, "Evidence-grounded verified agentic reasoning: A path toward eliminating LLM hallucination in empirical inference via tool-attested kernel proofs," arXiv:2607.12650, 2026 (ICML 2026 TAIGR workshop).

[19] B. Hall, S. Shome, and W. Eiers, "VeriFin: A neurosymbolic framework for verifying LLM-generated financial claims," arXiv:2608.10213, 2026.

[20] K. Patil, "RAGShield: Detecting numerical claim manipulation in government RAG systems," arXiv:2604.00387, 2026.

[21] M. Theologitis and D. Suciu, "Thucy: An LLM-based multi-agent system for claim verification across relational databases," AAAI 2026 Workshop on LLM-based Multi-Agent Systems, arXiv:2512.03278.

[22] F. A. Kamelhar, "GSAR: Typed grounding for hallucination detection and recovery in multi-agent LLMs," arXiv:2604.23366, 2026.

[23] L. G. Iyer, "Closed-world resolution against tool hallucination in LLM agents," arXiv:2609.19425, 2026.

[24] D. Cerda-Mardini, S. Chandar, and S. Madathil, "Consistent but miscalibrated: Evaluating LLM limitations for risk communication in natural language," arXiv:2607.03882, 2026.

[25] P. Balani and S. Panda, "Reporting under pressure: Separating factual and tonal sycophancy in LLM statistical analysis," arXiv:2609.27756, 2026.

[26] C. Thomson and E. Reiter, "A gold standard methodology for evaluating accuracy in data-to-text systems," in *Proc. 13th International Conference on Natural Language Generation (INLG)*, 2020.

[27] G. Yona *et al.*, "Can large language models faithfully express their intrinsic uncertainty in words?" in *Proc. EMNLP*, 2024.

[28] B. Wen *et al.*, "Know your limits: A survey of abstention in large language models," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 529–556, 2025.

[29] Y. Xu *et al.*, "A survey of large models in sports," in *Findings of ACL*, 2026, arXiv:2608.14377.

[30] S. Jamshidi, "Collective hallucination in multi-agent LLMs: Modeling and defense," arXiv:2606.07941, 2026.

[31] S. Lin, J. Hilton, and O. Evans, "Teaching models to express their uncertainty in words," *Transactions on Machine Learning Research*, 2022, arXiv:2205.14334.

[32] C. Wang, G. Szarvas, G. Balazs, P. Danchenko, and P. Ernst, "Calibrating verbalized probabilities for large language models," arXiv:2410.06707, 2024.

[33] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.

[34] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," arXiv:2309.11495, 2023.

[35] M. Bhatnagar, "FanCric: Multi-agentic framework for crafting fantasy 11 cricket teams," arXiv:2410.01307, 2024.

[36] P. Agarwal *et al.*, "CricBench: A multilingual benchmark for evaluating LLMs in cricket analytics," arXiv:2512.21877, version 3, 2026 (version 1: V. Devraj, D. Kumar, and J. S. Challa, 2025).

[37] H. Xia *et al.*, "SportQA: A benchmark for sports understanding in large language models," in *Proc. NAACL*, 2024.

[38] Y. Hu *et al.*, "SportsMetrics: Blending text and numerical data to understand information fusion in LLMs," in *Proc. ACL*, 2024.